

The Transformer Block

Attention and MLP, wired together and stacked · video 1:58

We have both halves of a layer: **attention**, where tokens share information, and the **MLP**, where each token processes it. This lesson wires them into a **transformer block**: each half **adds** its result to a stream of vectors, with a **layer norm** in front of it. Then the block is **stacked**, 12 times in GPT-2 small.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The residual stream	▶ 0:08-0:26	4	☐
2. Add, don't replace	▶ 0:27-0:49	4	☐
3. Layer norm	▶ 0:50-1:05	4	☐
4. Wiring the block	▶ 1:05-1:11	3	☐
5. Stack it	▶ 1:11-1:35	4	☐
6. The block in code	▶ 1:35-1:56	3	☐

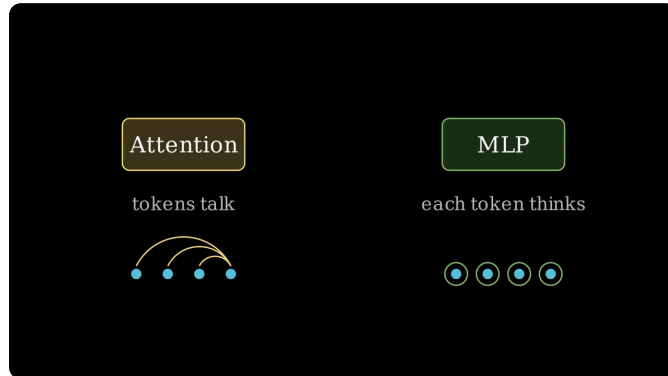
Before you start: this lesson builds on episodes 3–4 (each token becomes a vector, plus its position), 5–7 (attention) and 8 (the MLP). For the maths, Foundations F01 (Vectors), F04 (Straight Lines and Bends), F08 (Averages and Spread: the mean and standard deviation used by layer norm) and F11 (NumPy in Three Minutes, for the code in concept 6) help.

CONCEPT 1 OF 6

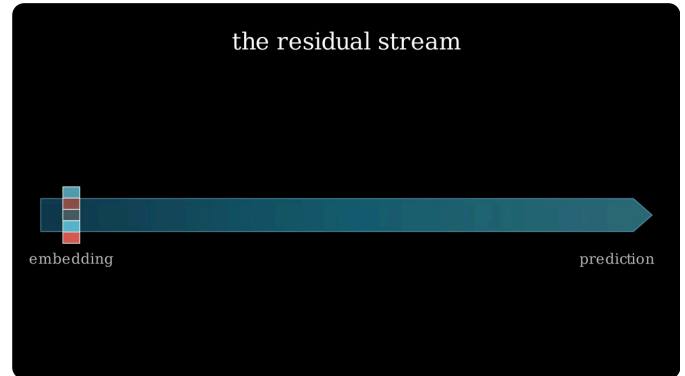
The residual stream

▶ Video 0:08-0:26

A layer has two halves: **attention**, where tokens share information, and the **MLP**, where each token processes it. The central idea for wiring them together is the **residual stream**.



▶ 0:14 The two halves: attention (tokens talk) and the MLP (each token thinks).



▶ 0:26 The residual stream: each token's vector flows from its embedding to the prediction.

Picture each token's vector flowing along a stream, from its **embedding** at the start to the **prediction** at the end. Every half along the way works on this vector (how, is concept 2).

KEY IDEA

Each token has one vector that **flows through the whole model**, from embedding to prediction: the **residual stream**. Attention and the MLP work on it along the way.

Check 1: The residual stream

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

What flows along the residual stream?

- A. Each token's vector
- B. The probabilities for the next word
- C. The text, as a list of words
- D. The attention weights of the first layer

1.2 PUT IN ORDER

Put these in the order a token's vector meets them: *prediction* · *embedding* · *attention and MLP halves*.

1.3 SHORT ANSWER

Which half lets tokens share information, and which processes each token on its own?

1.4 TRUE OR FALSE

"A text of 6 tokens has 6 vectors flowing along the residual stream, one per token."

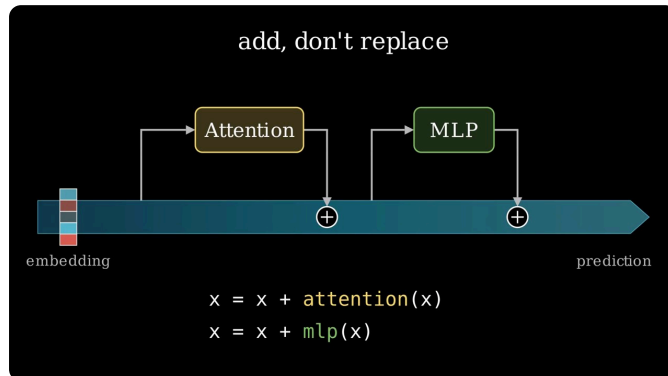
True ☐ False ☐

CONCEPT 2 OF 6

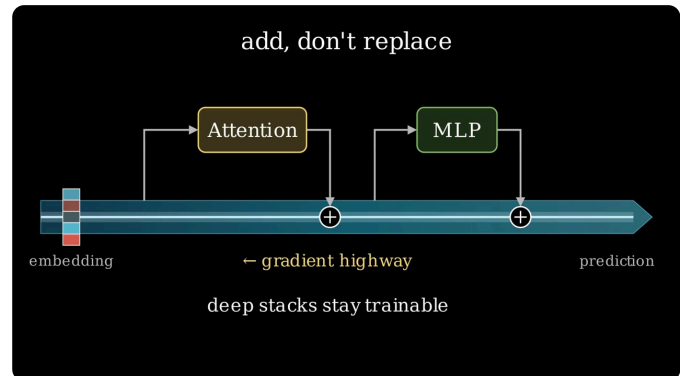
Add, don't replace

▶ Video 0:27-0:49

Attention and the MLP **don't replace** the vector. Each one **reads** from the stream, computes something, and **adds** its result back: $x = x + \text{attention}(x)$, then $x = x + \text{mlp}(x)$.



▶ 0:38 Each half reads from the stream, computes, and adds its result back at $\oplus$.



▶ 0:49 The unbroken stream is a direct path back through the network: a "gradient highway".

Adding matters. It **keeps the original information** around, so a half only has to add what is new, and it gives training a **direct path back** through the network. That is a big part of what makes **deep stacks** of layers trainable (training is episode 11).

KEY IDEA

Each half **adds** to the stream: $x = x + f(x)$. The original vector is never thrown away.

Check 2: Add, don't replace

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

$x = [1, 2]$. Attention returns $[0.5, -1]$; then the MLP returns $[0, 3]$. What is x after each step?

2.2 TRUE OR FALSE

"After the attention half, the stream holds only attention's output; the original vector is gone."

True ☐ False ☐

2.3 SHORT ANSWER

A half has learned nothing useful yet and always outputs zeros. What happens to x with $x = x + f(x)$? And with $x = f(x)$?

2.4 MULTIPLE CHOICE

Which two reasons does the video give for adding?

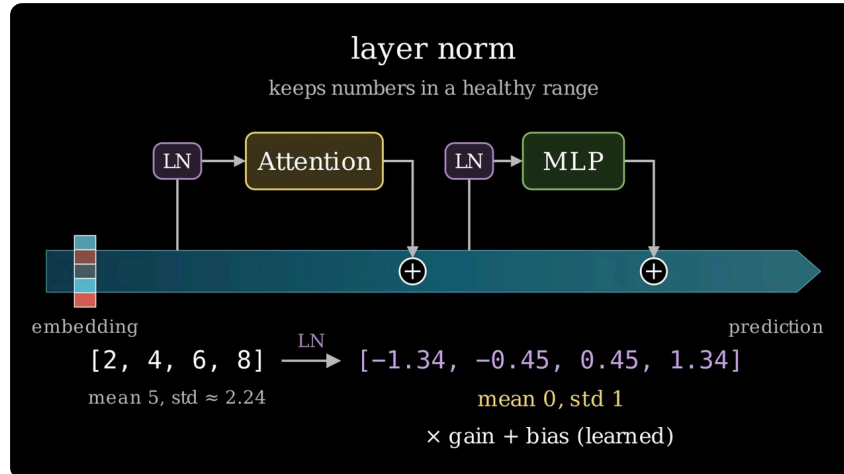
- A. It makes the model faster and smaller
- B. It keeps the original information, and gives training a direct path back through the network
- C. It turns vectors into probabilities and removes the need for training
- D. It lets the MLP see the other tokens

CONCEPT 3 OF 6

Layer norm

▶ Video 0:50–1:05

Before each half there is a **layer norm** (LN). It rescales a vector so its numbers have a **mean of 0** and a **standard deviation of 1** (subtract the mean, divide by the standard deviation), then applies a learned scale and shift: the **gain** and the **bias**.



▶ 1:04 A layer norm (LN) sits before each half. [2, 4, 6, 8] becomes mean 0, standard deviation 1, then gets a learned gain and bias.

Example: [2, 4, 6, 8] has mean 5. The squared distances from the mean are 9, 1, 1, 9, which average 5, so the standard deviation is $\sqrt{5} \approx 2.24$. Then $[-3, -1, 1, 3] / 2.24 = [-1.34, -0.45, 0.45, 1.34]$. This keeps the numbers in a healthy range as they flow through many layers.

KEY IDEA

$LN(x) = \text{gain} \times (x - \text{mean}) / \text{std} + \text{bias}$. Before the gain and bias, the output always has **mean 0 and standard deviation 1**.

Check 3: Layer norm

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Apply layer norm (gain 1, bias 0) to [1, 3].

3.2 SHORT ANSWER

Without calculating: what does layer norm (gain 1, bias 0) give for [12, 14, 16, 18]? And for [20, 40, 60, 80]?

3.3 CALCULATION

Apply a gain of 2 and a bias of 1 to $[-1.34, -0.45, 0.45, 1.34]$. Give each number to one decimal.

3.4 TRUE OR FALSE

“Layer norm is applied just once, at the start of the model.”

True ☐ False ☐

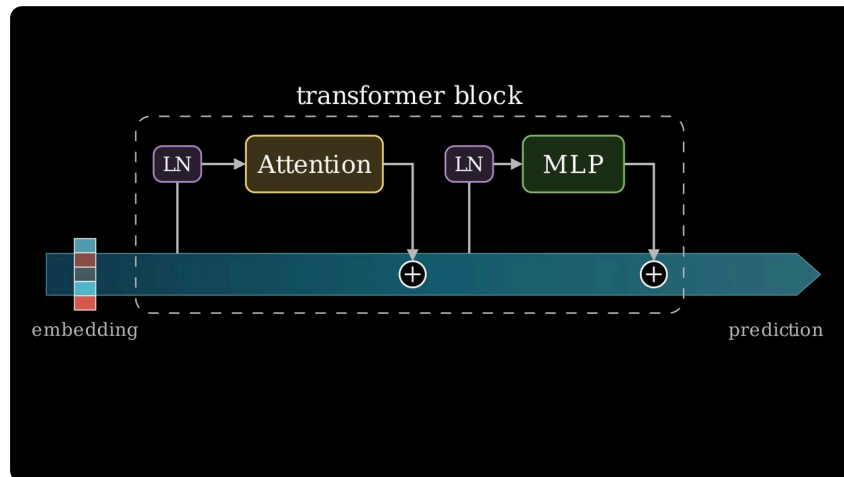
CONCEPT 4 OF 6

Wiring the block

▶ Video 1:05-1:11

Put it together and you have a **transformer block**: **layer norm, attention, add**; then **layer norm, MLP, add**. With the layer norms, the two lines are:

```
x = x + attention(layer_norm(x))    # tokens talk
x = x + mlp(layer_norm(x))         # each token thinks
```



▶ 1:10 One transformer block: layer norm, attention, add; layer norm, MLP, add.

Notice where the layer norm sits: it normalizes the **copy that goes into each half**, not the stream itself. The stream only ever has things **added** to it. (This arrangement is called **pre-norm**.)

KEY IDEA

Block = **LN** → **attention** → **add**, **LN** → **MLP** → **add**. Two halves, two layer norms, two additions.

Check 4: Wiring the block

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Put the six steps of one block in order: *add · MLP · layer norm · attention · add · layer norm*.

4.2 MULTIPLE CHOICE

Which line is the first half of the block?

- A. `x = attention(layer_norm(x))`
- B. `x = x + attention(layer_norm(x))`
- C. `x = layer_norm(x) + attention(x)`
- D. `x = x + layer_norm(x)`

4.3 TRUE OR FALSE

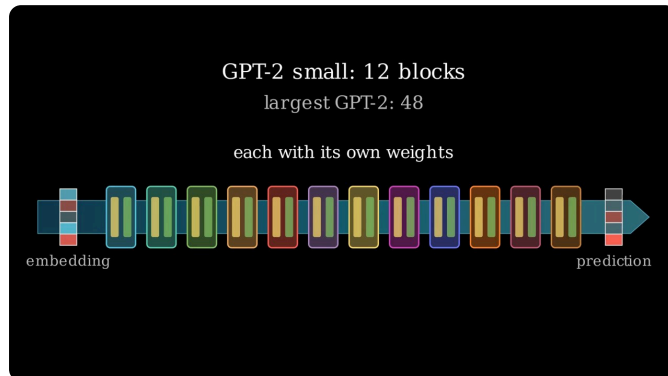
"The layer norm changes the vector stored in the residual stream."

True ☐ False ☐

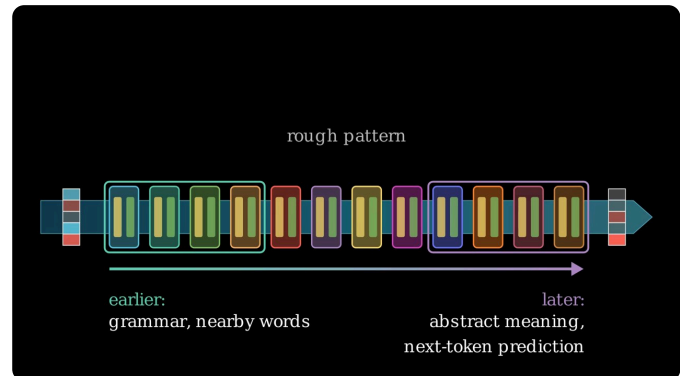
Stack it

▶ Video 1:11-1:35

Now **stack** the block. GPT-2 small has **12 blocks**; the largest GPT-2 has **48**. Every block has **its own weights**, and each one refines the vectors a little more as they flow along the stream.



▶ 1:22 GPT-2 small stacks 12 blocks, each with its own weights.



▶ 1:34 A rough pattern: earlier layers handle local things, later layers abstract meaning.

Researchers see a **rough pattern**. Earlier layers tend to handle **local** things, like grammar and nearby words. Later layers deal with more **abstract meaning**, and with **predicting the next token**.

KEY IDEA

The same design, repeated N times, with **different weights in every block**. Early blocks: local patterns. Late blocks: meaning and the next token.

Check 5: Stack it

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 TRUE OR FALSE

"The 12 blocks of GPT-2 small are copies of one block that share the same weights."

True ☐ False ☐

5.2 CALCULATION

From episode 8, one GPT-2 small layer has 4,718,592 MLP weights and 2,359,296 attention weights. About how many weights do the 12 blocks hold together (ignoring biases and layer norms)?

.....

5.3 CALCULATION

How many layer norms are inside GPT-2 small's 12 blocks? And inside the largest GPT-2's 48?

.....

5.4 MULTIPLE CHOICE

According to the rough pattern, which job is more typical of the later layers?

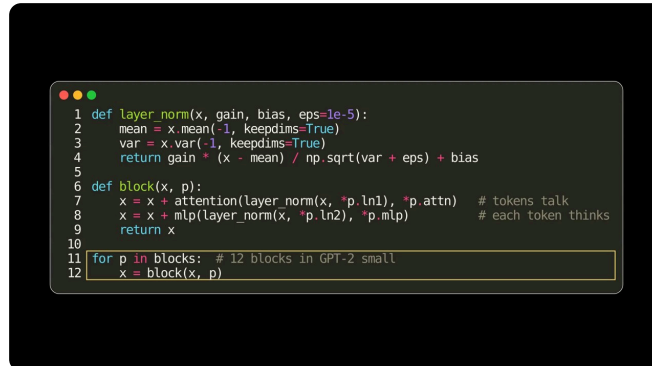
- A. Grammar and nearby words
- B. Abstract meaning and predicting the next token
- C. Splitting the text into tokens
- D. Looking up each token's embedding

CONCEPT 6 OF 6

The block in code

▶ Video 1:35-1:56

In code, a block is two lines: add attention of the normalized input, then add the MLP. The layer norm is a few more lines, and the **whole model is just a loop** over the blocks.



▶ 1:46 The code from the video: layer norm, a block, and the loop over all blocks.

```
def layer_norm(x, gain, bias, eps=1e-5):
    mean = x.mean(-1, keepdims=True)
    var = x.var(-1, keepdims=True)
    return gain * (x - mean) / np.sqrt(var + eps) + bias

def block(x, p):
    x = x + attention(layer_norm(x, *p.ln1), *p.attn) # tokens talk
    x = x + mlp(layer_norm(x, *p.ln2), *p.mlp) # each token thinks
    return x

for p in blocks: # 12 blocks in GPT-2 small
    x = block(x, p)
```

After the last block, each token's vector holds the model's view of **what comes next**. But it is still a vector: episode 10 turns it back into words.

KEY IDEA

for p in blocks: x = block(x, p) . The body of the model is one loop, and each block is two “normalize, compute, add” steps.

Check 6: The block in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 SHORT ANSWER

What would go wrong if you deleted `x +` from both lines of `block`?

6.2 TRUE OR FALSE

“After the last block, each token's vector is already a list of probabilities for the next word.”

True ☐ False ☐

6.3 HANDS-ON CODE

Try it yourself. Below is the video's `layer_norm` and a toy block whose attention has learned nothing yet (it returns zeros) and whose “MLP” just returns 0.1 times its input. (a) Predict the first two printed lines. (b) Predict the third. Why is it not all zeros, although attention returns zeros? (c) Delete `x +` from both lines of `block` and run again. What comes out, and why?

```
import numpy as np

def layer_norm(x, gain, bias, eps=1e-5):
    mean = x.mean(-1, keepdims=True)
    var = x.var(-1, keepdims=True)
    return gain * (x - mean) / np.sqrt(var + eps) + bias

def attention(z): return np.zeros_like(z)    # has learned nothing yet
def mlp(z):       return 0.1 * z             # a tiny toy "MLP"

def block(x):
    x = x + attention(layer_norm(x, 1.0, 0.0))
    x = x + mlp(layer_norm(x, 1.0, 0.0))
    return x

x = np.array([2.0, 4.0, 6.0, 8.0])
y = layer_norm(x, 1.0, 0.0)
print(np.round(y, 2), round(y.mean(), 3), round(y.std(), 3))
print(np.round(layer_norm(x, 2.0, 1.0), 2))
print(np.round(block(x), 3))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The residual stream • rewatch ▶ 0:08-0:26

- 1.1 A.** The stream carries each token's vector, from its embedding to the prediction.
- 1.2 embedding → attention and MLP halves → prediction.** The stream starts at the embedding and ends at the prediction; the halves sit along it.
- 1.3 Attention shares; the MLP processes each token alone.** Attention is where tokens talk; the MLP is where each token thinks (episode 8).
- 1.4 True.** Each token's vector flows along the stream, from its own embedding to the prediction.

2. Add, don't replace • rewatch ▶ 0:27-0:49

- 2.1 [1.5, 1], then [1.5, 4].** $x + \text{attention}(x) = [1 + 0.5, 2 - 1]$; then add $[0, 3]$.
- 2.2 False.** The output is added to x , so the original is still part of the sum.
- 2.3 Adding: x passes through unchanged. Replacing: x becomes all zeros.** Adding lets information flow past a half that has nothing to say; replacing wipes it out.
- 2.4 B.** Keeping the information and the direct path for training are why deep stacks can be trained.

3. Layer norm • rewatch ▶ 0:50-1:05

- 3.1 [-1, 1].** The mean is 2; both numbers are 1 away from it, so the standard deviation is 1; $(x - 2) / 1$.
- 3.2 Both give [-1.34, -0.45, 0.45, 1.34].** They are $[2, 4, 6, 8]$ shifted by 10 and scaled by 10. Subtracting the mean removes any shift; dividing by the standard deviation removes any scale.
- 3.3 [-1.7, 0.1, 1.9, 3.7].** $2 \times \text{value} + 1$. The learned gain and bias let the model pick its own scale and shift.
- 3.4 False.** There is one before each half: two in every block.

4. Wiring the block • rewatch ▶ 1:05-1:11

- 4.1 layer norm → attention → add → layer norm → MLP → add.** Each half: normalize a copy, compute, add the result to the stream.
- 4.2 B.** A replaces x instead of adding; C puts a normalized copy into the stream and feeds attention an un-normalized x ; D has no attention at all.
- 4.3 False.** It normalizes the copy fed into the half. Only the half's output is added to the stream.

5. Stack it • rewatch ▶ 1:11-1:35

- 5.1 False.** Same design, but every block has its own weights.
- 5.2 84,934,656, about 85 million.** $12 \times (4,718,592 + 2,359,296) = 12 \times 7,077,888$.
- 5.3 24 and 96.** Two per block, one before each half.
- 5.4 B.** A is typical of the earlier layers; C and D happen before the first block.

6. The block in code · rewatch ► 1:35-1:56

6.1 Each half would replace the vector instead of adding to it. The original information would be lost at every half, and training would lose its direct path back through the network (concept 2).

6.2 False. It is still a vector (768 numbers in GPT-2 small). Episode 10 turns it into probabilities.

6.3 (a) [-1.34 -0.45 0.45 1.34] 0.0 1.0 and [-1.68 0.11 1.89 3.68] · (b) [1.866 3.955 6.045 8.134] · (c) [0. 0. 0. 0.] (a) The example from concept 3: mean 0, standard deviation 1; then $2 \times \text{value} + 1$. (b) Attention adds zeros, so x passes through unchanged; the MLP then adds $0.1 \times [-1.34, -0.45, 0.45, 1.34]$: a small refinement of the original vector. (c) Without $x +$, attention replaces x with zeros, and nothing can bring the information back: layer norm of all zeros is zeros (the tiny `eps` avoids dividing by zero), and 0.1×0 is 0.